

1.

Welcome to the world of Java! In this challenge, we practice printing to stdout.

The code stubs in your editor declare a Solution class and a main method. Complete the main method by copying the two lines of code below and pasting them inside the body of your main method.

```
System.out.println("Hello, World.");
System.out.println("Hello, Java.");
```

Input Format

There is no input for this challenge.

Output Format

You must print two lines of output:

1. Print Hello, World. on the first line.
2. Print Hello, Java. on the second line.

Sample Output

```
Hello, World.
Hello, Java.
```

```
1 public class Solution {
2
3     public static void main(String[] args) {
4         /* Enter your code here. Print output to STDOUT. Your class should be named
5         Solution. */
6         System.out.println("Hello, World.");
7         System.out.println("Hello, Java.");
8     }
9 }
```

2.

Most HackerRank challenges require you to read input from [stdin](#) (standard input) and write output to [stdout](#) (standard output).

One popular way to read input from stdin is by using the [Scanner class](#) and specifying the Input Stream as System.in. For example:

```
Scanner scanner = new Scanner(System.in);
String myString = scanner.next();
int myInt = scanner.nextInt();
scanner.close();
```

The code above creates a Scanner object named `scanner` and uses it to read a String and an int. It then closes the Scanner object because there is no more input to read, and prints to stdout using `System.out.println(String)`. So, if our input is:

```
Hi 5
```

Our code will print:

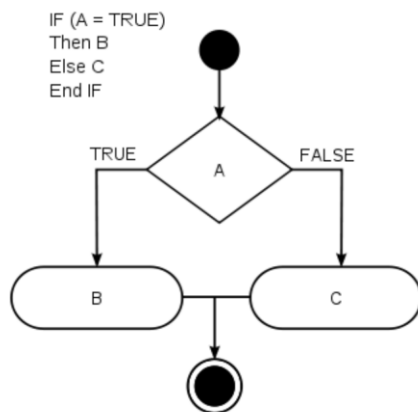
```
myString is: Hi
myInt is: 5
```

Alternatively, you can use the [BufferedReader class](#).

```
1 import java.util.*;
2
3 public class Solution {
4
5     public static void main(String[] args) {
6         Scanner scan = new Scanner(System.in);
7         int a = scan.nextInt();
8         int b = scan.nextInt();
9         int c = scan.nextInt();
10        // Complete this line
11        // Complete this line
12
13        System.out.println(a);
14        System.out.println(b);
15        System.out.println(c);
16        // Complete this line
17        // Complete this line
18    }
19 }
20
```

3.

In this challenge, we test your knowledge of using if-else conditional statements to automate decision-making processes. An if-else statement has the following logical flow:



Source: Wikipedia

```
9 public class Solution {
10
11     public static void main(String[] args) {
12
13         Scanner sc=new Scanner(System.in);
14         int n=sc.nextInt();
15         if(n%2!=0){
16             System.out.println("Weird");
17         }
18         else{
19             if(n>2 &&n<=5){
20                 System.out.println("Not Weird");
21             }
22             else if(n>=6 &&n<=20){
23                 System.out.println("Weird");
24             }
25             else{
26                 System.out.println("Not Weird");
27             }
28         }
29         sc.close();
30     }
31 }
32
33
34
```

4.

Problem

In this challenge, you must read an integer, a double, and a String from stdin, then print the values according to the instructions in the Output Format section below. To make the problem a little easier, a portion of the code is provided for you in the editor.

Note: We recommend completing [Java Stdin and Stdout I](#) before attempting this challenge.

Input Format

There are three lines of input:

1. The first line contains an integer.
2. The second line contains a double.
3. The third line contains a String.

Output Format

There are three lines of output:

1. On the first line, print `String`: followed by the unaltered String read from stdin.
2. On the second line, print `Double`: followed by the unaltered double read from stdin.
3. On the third line, print `Int`: followed by the unaltered integer read from stdin.

To make the problem easier, a portion of the code is already provided in the editor.

Note: If you use the `nextLine()` method immediately following the `nextInt()` method, recall that `nextInt()` reads integer tokens; because of this, the last newline character for that line of integer input is still queued in the input buffer and the next `nextLine()` will be reading the remainder of the integer line (which is empty).

Sample Input

```

1  import java.util.Scanner;
2
3  public class Solution {
4
5      public static void main(String[] args) {
6          Scanner scan = new Scanner(System.in);
7          int i = scan.nextInt();
8
9          double d = scan.nextDouble();
10         scan.nextLine();
11         String s = scan.nextLine();
12
13         // Write your code here.
14
15         System.out.println("String: " + s);
16         System.out.println("Double: " + d);
17         System.out.println("Int: " + i);
18     }
19
20 }
21

```

Line: 21 Col: 1

[Upload Code as File](#) ☐ Test against custom input [Run Code](#) [Submit Code](#)

5.

Java's `System.out.printf` function can be used to print formatted output. The purpose of this exercise is to test your understanding of formatting output using `printf`.

To get you started, a portion of the solution is provided for you in the editor; you must format and print the input to complete the solution.

Input Format

Every line of input will contain a String followed by an integer.

Each String will have a maximum of 10 alphabetic characters, and each integer will be in the inclusive range from 0 to 999.

Output Format

In each line of output there should be two columns:

The first column contains the String and is left justified using exactly 15 characters.

The second column contains the integer, expressed in exactly 3 digits; if the original input has less than three digits, you must pad your output's leading digits with zeroes.

Sample Input

```

java 100
cpp 65
python 50

```

Sample Output

```

=====

```

```

1  import java.util.Scanner;
2
3  public class Solution {
4
5      public static void main(String[] args) {
6          Scanner sc=new Scanner(System.in);
7          System.out.println("=====");
8          for(int i=0;i<3;i++){
9              String s1=sc.next();
10             int x=sc.nextInt();
11             //Complete this line
12             System.out.printf("%-15s%03d\n", s1, x);
13         }
14         System.out.println("=====");
15     }
16
17 }
18
19
20
21

```

Line: 21 Col: 1

[Upload Code as File](#) ☐ Test against custom input [Run Code](#) [Submit Code](#)

6.

Objective

In this challenge, we're going to use loops to help us do some simple math.

Task

Given an integer, N , print its first 10 multiples. Each multiple $N \times i$ (where $1 \leq i \leq 10$) should be printed on a new line in the form: $N \times i = \text{result}$.

Input Format

A single integer, N .

Constraints

- $2 \leq N \leq 20$

Output Format

Print 10 lines of output; each line i (where $1 \leq i \leq 10$) contains the **result** of $N \times i$ in the form:

$N \times i = \text{result}$.

Sample Input

```
2
```

Sample Output

```
2 x 1 = 2
2 x 2 = 4
...
```

Change Theme Language Java 7

```

1 import java.io.*;
2 import java.math.*;
3 import java.security.*;
4 import java.text.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.regex.*;
8
9
10
11 public class Solution {
12     public static void main(String[] args) throws IOException {
13         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(
14             System.in));
15
16         int N = Integer.parseInt(bufferedReader.readLine().trim());
17
18         for(int i=1;i<=10;i++){
19             System.out.println(N+" x "+i+" = "+N*i);
20         }
21         bufferedReader.close();
22     }
23 }
24

```

Line: 24 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

7.

We use the integers a , b , and n to create the following series:

$$(a + 2^0 \cdot b), (a + 2^0 \cdot b + 2^1 \cdot b), \dots, (a + 2^0 \cdot b + 2^1 \cdot b + \dots + 2^{n-1} \cdot b)$$

You are given q queries in the form of a , b , and n . For each query, print the series corresponding to the given a , b , and n values as a single line of n space-separated integers.

Input Format

The first line contains an integer, q , denoting the number of queries. Each line i of the q subsequent lines contains three space-separated integers describing the respective a_i , b_i , and n_i values for that query.

Constraints

- $0 \leq q \leq 500$
- $0 \leq a, b \leq 50$
- $1 \leq n \leq 15$

Output Format

For each query, print the corresponding series on a new line. Each series must be printed in order as a single line of n space-separated integers.

Sample Input

```
2
0 2 10
```

Change Theme Language Java 7

```

1 import java.util.*;
2 import java.io.*;
3
4 class Solution{
5     public static void main(String []argh){
6         Scanner in = new Scanner(System.in);
7         int t=in.nextInt();
8         for(int i=0;i<t;i++){
9             int a = in.nextInt();
10            int b = in.nextInt();
11            int n = in.nextInt();
12            int currentSum = a;
13
14            for (int j = 0; j < n; j++) {
15                currentSum += (1 << j) * b;
16
17                System.out.print(currentSum + " ");
18            }
19            System.out.println();
20        }
21        in.close();
22    }
23 }
24

```

Line: 24 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

8.

"In computing, End Of File (commonly abbreviated EOF) is a condition in a computer operating system where no more data can be read from a data source." — (Wikipedia: End-of-file)

The challenge here is to read n lines of input until you reach EOF, then number and print all n lines of content.

Hint: Java's Scanner.hasNext() method is helpful for this problem.

Input Format

Read some unknown n lines of input from stdin(System.in) until you reach EOF; each line of input contains a non-empty String.

Output Format

For each line, print the line number, followed by a single space, and then the line content received as input.

Sample Input

```
Hello world
I am a file
Read me until end-of-file.
```

Sample Output

```
1 Hello world
2 I am a file
3 Read me until end-of-file.
```

Change Theme Language Java 7

```

1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 public class Solution {
8
9     public static void main(String[] args) {
10         Scanner scan=new Scanner(System.in);
11         int line=1;
12         while(scan.hasNext()){
13             System.out.println(line+" "+scan.nextLine());
14             line++;
15         }
16
17         /* Enter your code here. Read input from STDIN. Print output to
18            STDOUT. Your class should be named Solution. */
19     }
20 }
21

```

Line: 19 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

9.

Static initialization blocks are executed when the class is loaded, and you can initialize static variables in those blocks.

It's time to test your knowledge of Static initialization blocks. You can read about it [here](#).

You are given a class `Solution` with a main method. Complete the given code so that it outputs the area of a parallelogram with breadth B and height H . You should read the variables from the standard input.

If $B \leq 0$ or $H \leq 0$, the output should be "java.lang.Exception: Breadth and height must be positive" without quotes.

Input Format

There are two lines of input. The first line contains B : the breadth of the parallelogram. The next line contains H : the height of the parallelogram.

Constraints

- $-100 \leq B \leq 100$
- $-100 \leq H \leq 100$

Output Format

If both values are greater than zero, then the main method must output the area of the parallelogram. Otherwise, print "java.lang.Exception: Breadth and height must be positive" without quotes.

Sample input 1

```

1 > import java.io.*; ...
8 static int B;
9 static int H;
10 static boolean flag = false;
11 static {
12     Scanner sc = new Scanner(System.in);
13     B = sc.nextInt();
14     H = sc.nextInt();
15     sc.close();
16
17     if (B > 0 && H > 0) {
18         flag = true;
19     } else {
20         System.out.println("java.lang.Exception: Breadth and height must be positive");
21     }
22 }
23
24 > public static void main(String[] args) { ...

```

Line: 8 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

10.

You are given an integer n , you have to convert it into a string.

Please complete the partially completed code in the editor. If your code successfully converts n into a string s the code will print "Good job". Otherwise it will print "Wrong answer".

n can range between -100 to 100 inclusive.

Sample Input 0

100

Sample Output 0

Good job

```

4 import java.util.*; ...
13 String s = Integer.toString(n);
14
15 if (n == Integer.parseInt(s)) {
16     System.out.println("Good job");
17 } else {
18     System.out.println("Wrong answer.");
19 }
20 } catch (DoNotTerminate.ExitTrappedException e) {
21     System.out.println("Unsuccessful Termination!!");
22 }
23 }
24 }
25
26 //The following class will prevent you from terminating the code using exit(0)!
27 class DoNotTerminate {
28
29     public static class ExitTrappedException extends SecurityException {
30
31         private static final long serialVersionUID = 1;
32     }
33
34     public static void forbidExit() {
35         final SecurityManager securityManager = new SecurityManager() {
36             @Override

```

Line: 13 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

11.

"A string is traditionally a sequence of characters, either as a literal constant or as some kind of variable." — Wikipedia: [String \(computer science\)](#)

This exercise is to test your understanding of Java Strings. A sample String declaration:

```
String myString = "Hello World!"
```

The elements of a String are called characters. The number of characters in a String is called the length, and it can be retrieved with the `String.length()` method.

Given two strings of lowercase English letters, A and B , perform the following operations:

- Sum the lengths of A and B .
- Determine if A is lexicographically larger than B (i.e.: does B come before A in the dictionary?).
- Capitalize the first letter in A and B and print them on a single line, separated by a space.

Input Format

The first line contains a string A . The second line contains another string B . The strings are comprised of only lowercase English letters.

Output Format

There are three lines of output:

For the first line, sum the lengths of A and B .

```

1 import java.io.*;
2 import java.util.*;
3 import java.lang.String;
4 public class Solution {
5
6     public static void main(String[] args) {
7
8         Scanner sc = new Scanner(System.in);
9         String A = sc.next();
10        String B = sc.next();
11        int len1 = A.length();
12        int len2 = B.length();
13        System.out.println(len1 + len2);
14        if (A.compareTo(B) > 0) {
15            System.out.println("Yes");
16        } else {
17            System.out.println("No");
18        }
19
20        System.out.println(A.substring(0, 1).toUpperCase() + A.substring(1) + " " +
21                           B.substring(0, 1).toUpperCase() + B.substring(1));
22        sc.close();
23    }
24 }

```

Line: 27 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

12.

Given a string, *s*, and two indices, *start* and *end*, print a **substring** consisting of all characters in the inclusive range from *start* to *end* - 1. You'll find the String class' **substring method** helpful in completing this challenge.

Input Format

The first line contains a single string denoting *s*.

The second line contains two space-separated integers denoting the respective values of *start* and *end*.

Constraints

- $1 \leq |s| \leq 100$
- $0 \leq start < end \leq n$
- String *s* consists of English alphabetic letters (i.e., [a - zA - Z]) only.

Output Format

Print the substring in the inclusive range from *start* to *end* - 1.

Sample Input

```
Helloworld
3 7
```

Sample Output

```
lowo
```

Change Theme Language Java 7

```

1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 public class Solution {
8
9     public static void main(String[] args) {
10         Scanner in = new Scanner(System.in);
11         String S = in.next();
12         int start = in.nextInt();
13         int end = in.nextInt();
14         System.out.println(S.substring(start,end));
15     }
16 }
17

```

Line: 17 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

13.

We define the following terms:

- Lexicographical Order**, also known as alphabetic or dictionary order, orders characters as follows:

$$A < B < \dots < Y < Z < a < b < \dots < y < z$$

For example, ball < cat, dog < dorm, Happy < happy, Zoo < ball.

- A **substring** of a string is a contiguous block of characters in the string. For example, the substrings of abc are a, b, c, ab, bc, and abc.

Given a string, *s*, and an integer, *k*, complete the function so that it finds the lexicographically smallest and largest substrings of length *k*.

Function Description

Complete the getSmallestAndLargest function in the editor below.

getSmallestAndLargest has the following parameters:

- string *s*: a string
- int *k*: the length of the substrings to find

Returns

- string: the string ' + "\n" + ' where and are the two substrings

Input Format

The first line contains a string denoting *s*.

The second line contains an integer denoting *k*.

Constraints

Change Theme Language Java 7

```

1 > import java.util.Scanner; ...
4
5
6 public static String getSmallestAndLargest(String s, int k) {
7
8     String smallest = s.substring(0, k);
9     String largest = s.substring(0, k);
10    for (int i = 1; i <= s.length() - k; i++) {
11        String currentSubString = s.substring(i, i + k);
12
13        if (currentSubString.compareTo(smallest) < 0) {
14            smallest = currentSubString;
15        }
16
17        if (currentSubString.compareTo(largest) > 0) {
18            largest = currentSubString;
19        }
20    }
21
22    return smallest + "\n" + largest;
23 }
24
25 > ...
26

```

Line: 4 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

14.

A palindrome is a word, phrase, number, or other sequence of characters which reads the same backward or forward.

Given a string *A*, print Yes if it is a palindrome, print No otherwise.

Constraints

- A* will consist at most 50 lower case english letters.

Sample Input

```
madam
```

Sample Output

```
Yes
```

Change Theme Language Java 7

```

1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7
8         Scanner sc=new Scanner(System.in);
9         String str=sc.next();
10        /* Enter your code here. Print output to STDOUT. */
11        int left = 0;
12        int right = str.length() - 1;
13        boolean isPalindrome = true;
14        while (left < right) {
15            if (str.charAt(left) != str.charAt(right)) {
16                isPalindrome = false;
17                break;
18            }
19            left++;
20            right--;
21        }
22
23        if (isPalindrome) {
24            System.out.println("Yes");
25        }
26    }
27 }
28

```

Line: 36 Col: 1

15.

Two strings, a and b , are called anagrams if they contain all the same characters in the same frequencies. For this challenge, the test is not case-sensitive. For example, the anagrams of CAT are CAT, ACT, tac, TCA, aTC, and CtA.

Function Description

Complete the isAnagram function in the editor.

isAnagram has the following parameters:

- string a : the first string
- string b : the second string

Returns

- boolean: If a and b are case-insensitive anagrams, return true. Otherwise, return false.

Input Format

The first line contains a string a .

The second line contains a string b .

Constraints

- $1 \leq \text{length}(a), \text{length}(b) \leq 50$
- Strings a and b consist of English alphabetic characters.
- The comparison should NOT be case sensitive.

Sample Input 0

```
anagram
margana
```

```

1 > import java.util.Scanner;
2
3
4 static boolean isAnagram(String a, String b) {
5     if (a == null || b == null || a.length() != b.length()) {
6         return false;
7     }
8     a = a.toLowerCase();
9     b = b.toLowerCase();
10    int[] frequencies = new int[26];
11
12    for (int i = 0; i < a.length(); i++) {
13        frequencies[a.charAt(i) - 'a']++;
14        frequencies[b.charAt(i) - 'a']--;
15    }
16
17    for (int count : frequencies) {
18        if (count != 0) {
19            return false;
20        }
21    }
22    return true;
23 }
24
25
26
27

```

Line: 4 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

16.

Using **Regex**, we can easily match or search for patterns in a text. Before searching for a pattern, we have to specify one using some well-defined syntax.

In this problem, you are given a pattern. You have to check whether the syntax of the given pattern is valid.

Note: In this problem, a regex is only valid if you can compile it using the `Pattern.compile` method.

Input Format

The first line of input contains an integer N , denoting the number of test cases. The next N lines contain a string of any printable characters representing the pattern of a regex.

Output Format

For each test case, print `Valid` if the syntax of the given pattern is correct. Otherwise, print `Invalid`. Do not print the quotes.

Sample Input

```
3
([A-Z])(.+)
[AZ[a-z](a-z)
batcatpat(nat
```

Sample Output

```
Valid
Invalid
Invalid
```

```

1 import java.util.Scanner;
2 import java.util.regex.Pattern;
3 import java.util.regex.PatternSyntaxException;
4
5 public class Solution {
6     public static void main(String[] args) {
7         Scanner in = new Scanner(System.in);
8
9
10        if (in.hasNextInt()) {
11            int testCases = in.nextInt();
12            in.nextLine();
13
14
15            while (testCases > 0) {
16                String pattern = in.nextLine();
17
18                try {
19                    Pattern.compile(pattern);
20                    System.out.println("Valid");
21                } catch (PatternSyntaxException e) {
22                    System.out.println("Invalid");
23                }
24            }
25        }
26    }
27 }

```

Line: 33 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

17.

Problem

Write a class called `MyRegex` which will contain a string pattern. You need to write a regular expression and assign it to the pattern such that it can be used to validate an IP address. Use the following definition of an IP address:

IP address is a string in the form "A.B.C.D", where the value of A

Some valid IP address:

```
000.12.12.034
121.234.12.12
23.45.12.56
```

Some invalid IP address:

```
000.12.234.23.23
666.666.23.23
213.123.23.32
23.45.22.32
I.Am.not.an.ip
```

In this problem you will be provided strings containing any combination of ASCII characters. You have to write a regular expression to find the valid IPs.

Just write the `MyRegex` class which contains a String **pattern**. The string should contain the correct regular expression.

(`MyRegex` class MUST NOT be public)

Change Theme
Language: Java 7

```

1 > import java.util.regex.Matcher; ...
16 class MyRegex {
17     private static final String bytePattern = "([01]?\\d{1,2}|2[0-4]\\d|25[0-5])"
18     public final String pattern = "A" + bytePattern + "\\." + bytePattern + "\\." + bytePattern + "\\." + bytePattern + "$";
19 }
20 //Write your code here
21

```

Line: 16 Col: 1

Upload Code as File
Test against custom input
Run Code
Submit Code

18.

Java's `BigDecimal` class can handle arbitrary-precision signed decimal numbers. Let's test your knowledge of them!

Given an array, s , of n real number strings, sort them in descending order — but wait, there's more! Each number must be printed in the exact same format as it was read from stdin, meaning that `.1` is printed as `.1` and `0.1` is printed as `0.1`. If two numbers represent numerically equivalent values (e.g., `.1` $\equiv$ `0.1`), then they must be listed in the same order as they were received as input).

Complete the code in the unlocked section of the editor below. You must rearrange array s 's elements according to the instructions above.

Input Format

The first line consists of a single integer, n , denoting the number of integer strings. Each line i of the n subsequent lines contains a real number denoting the value of s_i .

Constraints

- $1 \leq n \leq 200$
- Each s_i has at most 300 digits.

Output Format

Locked stub code in the editor will print the contents of array s to stdout. You are only responsible for reordering the array's elements.

Sample Input

Change Theme
Language: Java 7

```

1 > import java.math.BigDecimal; ...
14
15 //Write your code here
16 // Sort only the first 'n' elements using a custom Comparator
17 Arrays.sort(s, 0, n, new Comparator<String>() {
18     @Override
19     public int compare(String a, String b) {
20         // Convert strings to BigDecimal for accurate numerical compariso
21         BigDecimal bdA = new BigDecimal(a);
22         BigDecimal bdB = new BigDecimal(b);
23
24         // Compare b to a for a descending order sort
25         return bdB.compareTo(bdA);
26     }
27 });
28 //Output...

```

Line: 14 Col: 1

19.

An array is a simple data structure used to store a collection of data in a contiguous block of memory. Each element in the collection is accessed using an index, and the elements are easy to find because they're stored sequentially in memory.

Because the collection of elements in an array is stored as a big block of data, we typically use arrays when we know exactly how many pieces of data we're going to have. For example, you might use an array to store a list of student ID numbers, or the names of state capitals. To create an array of integers named ***myArray*** that can hold four integer values, you would write the following code:

```
int[] myArray = new int[4];
```

This sets aside a block of memory that's capable of storing 4 integers. Each integer storage cell is assigned a unique index ranging from 0 to one less than the size of the array, and each cell initially contains a 0. In the case of ***myArray***, we can store integers at indices 0, 1, 2, and 3. Let's say we wanted the last cell to store the number 12; to do this, we write:

```
myArray[3] = 12;
```

Similarly, we can print the contents of the last cell with the following code:

```
System.out.println(myArray[3]);
```

The code above prints the value stored at index 3 of ***myArray***, which is 12 (the value we

Change Theme
Language: Java 7

```

5 import java.util.*; ...
9 int a[]=new int[n];
10 for (int i = 0; i < n; i++) {
11     a[i] = scan.nextInt();
12 }
13
14
15 scan.close();
16
17 // Prints each sequential element in array a
18 for (int i = 0; i < a.length; i++) {
19     System.out.println(a[i]);
20 }
21
22

```

Line: 15 Col: 22

Upload Code as File
Test against custom input
Run Code
Submit Code

20.

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The `Player` class is provided for you in your editor. It has 2 fields: a `name` String and a `score` integer.

Given an array of `n` `Player` objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the `Solution` class.

The first line contains an integer, `n`, denoting the number of players.

Each of the `n` subsequent lines contains a player's `name` and `score`, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2 players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in `Solution` will create a `Checker` object, use it to sort the `Player` array, and print each sorted element.

```
Change Theme Language Java 7

1 import java.util.*;
2
3 // Write your Checker class here
4 class Checker implements Comparator<Player> {
5     @Override
6     public int compare(Player a, Player b) {
7         // 1. If scores are equal, sort alphabetically by name (ascending)
8         if (a.score == b.score) {
9             return a.name.compareTo(b.name);
10        }
11
12        // 2. Otherwise, sort by score in descending order
13        // Returns a positive number if b.score > a.score (places 'b' before 'a')
14        return Integer.compare(b.score, a.score);
15    }
16 }
17 > class Player{...
```

Line: 2 Col: 1

Upload Code as File Test against custom input RunCode SubmitCode

21.

You are given a class `Solution` and an inner class `Inner.Private`. The main method of class `Solution` takes an integer `num` as input. The `powerof2` in class `Inner.Private` checks whether a number is a power of 2. You have to call the method `powerof2` of the class `Inner.Private` from the main method of the class `Solution`.

Constraints

$$1 \leq \text{num} \leq 2^{30}$$

Sample Input

8

Sample Output

8 is power of 2
An instance of class: Solution.Inner.Private has been created

```
Change Theme Language Java 7

1 > import java.io.*;...
17
18 Solution.Inner innerInstance = new Solution.Inner();
19 o = innerInstance.new Private();
20 System.out.println(num + " is " + ((Solution.Inner.Private) o).powerOf2(num));
21
22 System.out.println("An instance of class: " + o.getClass().getCanonicalName() + " has been created");
23
24 //end of try
25
26 catch (DoNotTerminate.ExitTrappedException e) {
27     System.out.println("Unsuccessful Termination!!");
28 }
29
30 //end of main
31 static class Inner{
32     private class Private{
33         private String powerof2(int num){
34             return ((num&num-1)==0)?"power of 2":"not a power of 2";
35         }
36     }
37 }
38 //end of Solution
```

Line: 22 Col: 184

22.

MD5 (Message-Digest algorithm 5) is a widely-used cryptographic hash function with a 128-bit hash value. Here are some common uses for MD5:

- To store a one-way hash of a password.
- To provide some assurance that a transferred file has arrived intact.

MD5 is one in a series of message digest algorithms designed by Professor Ronald Rivest of MIT (Rivest, 1994); however, the security of MD5 has been severely compromised, most infamously by the Flame malware in 2012. The CMU Software Engineering Institute essentially considers MD5 to be "cryptographically broken and unsuitable for further use".

Given an alphanumeric string, `s`, denoting a password, compute and print its MD5 encryption value.

Input Format

A single alphanumeric string denoting `s`.

Constraints

- $6 \leq |s| \leq 20$
- String `s` consists of English alphabetic letters (i.e., `[a-zA-Z]`) and/or decimal digits (i.e., 0 through 9) only.

Output Format

Print the MD5 encryption value of `s` on a new line.

Sample Input 0

HelloWorld

```
Change Theme Language Java 7

1 import java.util.Scanner;
2 import java.security.MessageDigest;
3 import java.security.NoSuchAlgorithmException;
4
5 public class Solution {
6
7     public static void main(String[] args) {
8         Scanner scan = new Scanner(System.in);
9         if (scan.hasNext()) {
10             String s = scan.next();
11
12             try {
13                 MessageDigest md = MessageDigest.getInstance("MD5");
14                 byte[] hashBytes = md.digest(s.getBytes());
15
16                 StringBuilder hexString = new StringBuilder();
17                 for (byte b : hashBytes) {
18                     String hex = Integer.toHexString(0xff & b);
19                     if (hex.length() == 1) {
20                         hexString.append('0');
21                     }
22                     hexString.append(hex);
23                 }
24                 System.out.println(hexString.toString());
25             }
26         }
27     }
28 }
```

Line: 33 Col: 1

Upload Code as File Test against custom input RunCode SubmitCode